Maintenance and Support Plan

P2P Car Rental System

Course	MSE800 Professional Software Engineering
Assignment	Assignment 1 - Object-Oriented Programming
Task	Task 3 - Software Evolution (30% weighting)
Document	Maintenance and Support Plan
Version	1.0.0
Date	May 2026

This document outlines the long-term maintenance and support strategy for the P2P Car Rental System, including software maintenance, versioning, and backward compatibility planning.

1. Executive Summary

Software is never "finished" — it requires continuous care after release. This Maintenance and Support Plan outlines how the P2P Car Rental System will be maintained, evolved, and supported throughout its lifecycle. A well-planned maintenance strategy is critical for long-term success: industry research shows that 60-80% of a software product's total cost is spent **after** the initial release, not during development.

This document addresses three core areas required by the assessment rubric:

1. **Software Maintenance** — How we handle updates, bug fixes, and new features.
2. **Versioning** — How we number and track releases.
3. **Backward Compatibility** — How we ensure existing users and data remain functional.

> **Why this matters:** Without a maintenance plan, a P2P platform like ours would quickly degrade — bugs would accumulate, security vulnerabilities would go unpatched, and incompatible changes would break existing users' bookings. A disciplined approach protects both our users (renters and owners) and our platform reputation.

2. Software Maintenance Strategy

Software maintenance is classified into four categories by the IEEE standard (IEEE Std 14764-2006). Our plan addresses all four:

Type	Description	Examples in Our System
Corrective	Fixing defects discovered after release.	Booking date overlap bug; password reset email not sending; pricing calculation off-by-one error.
Adaptive	Adapting to changes in the environment (OS, libraries, regulations).	Upgrading from Python 3.8 to 3.12; migrating from SQLite to PostgreSQL when user base grows; complying with new GDPR or NZ Privacy Act amendments.
Perfective	Improving performance, usability, or maintainability without changing functionality.	Adding database indexes for faster searches; refactoring the booking service; improving CLI feedback messages.
Preventive	Reducing the chance of future failures by improving the codebase.	Adding unit tests; updating dependencies before security advisories; refactoring complex methods into smaller, testable units.

2.1 Issue Handling Workflow

When a bug or feature request comes in, we follow a structured workflow:

Step	Action	Responsible
1	Issue is reported via GitHub Issues or in-app feedback form	User / QA
2	Triage: assign severity (Critical / High / Medium / Low)	Maintainer
3	Reproduce the bug locally; document steps	Developer
4	Create a fix branch: bugfix/issue-NNN	Developer
5	Write a unit test that fails on the bug	Developer
6	Implement the fix until the test passes	Developer
7	Open Pull Request; peer code review	Developer + Reviewer
8	Merge to main; deploy in next patch release	Maintainer
9	Update CHANGELOG.md; notify affected users	Maintainer

2.2 Severity Levels and Response Times

Not all bugs are equal. We commit to the following Service Level Agreement (SLA):

Severity	Definition	Response Time	Fix Target
Critical	System unusable; data loss; security breach	Within 4 hours	24 hours
High	Major feature broken (e.g. cannot book a car)	Within 1 business day	3 business days
Medium	Minor feature impacted; workaround exists	Within 3 business days	Next scheduled release
Low	Cosmetic issues; minor inconveniences	Within 1 week	Future release

2.3 Update Release Schedule

We follow a predictable, cadence-based release schedule. Predictability allows users to plan around releases:

Release Type	Frequency	Contains
Patch (1.0.x)	As needed (typically weekly)	Bug fixes, security patches. No new features. Fully backward compatible.
Minor (1.x.0)	Every 4-6 weeks	New features, performance improvements. Fully backward compatible.
Major (x.0.0)	Every 12-18 months	Breaking changes allowed. Migration guide provided. Old version supported for 6 months after major release.

2.4 Continuous Improvement Practices

Code reviews. Every pull request requires at least one reviewer approval. No exceptions, even for the lead developer.

Automated testing. Unit tests run on every commit via GitHub Actions. Pull requests cannot merge if tests fail.

Dependency monitoring. Tools like Dependabot scan for outdated or vulnerable packages weekly and open auto-PRs to update them.

Code quality metrics. Static analysis (pylint, mypy, flake8) runs on every commit. Quality must not regress below thresholds.

User feedback loop. Monthly review of GitHub Issues and in-app feedback to identify common pain points.

Documentation as code. README, API docs, and inline comments are kept up-to-date alongside code changes — never as an afterthought.

3. Versioning Strategy

A clear versioning scheme allows users to understand at a glance what kind of change a new release contains. We adopt **Semantic Versioning 2.0.0** (semver.org), the industry standard.

3.1 Semantic Versioning (SemVer)

Every version number has three parts: `MAJOR.MINOR.PATCH` — for example, `1.4.2`.

Component	Increment When...	Example
MAJOR	You make incompatible API or data-schema changes that require user action.	1.4.2 → 2.0.0
MINOR	You add new functionality in a backward-compatible manner.	1.4.2 → 1.5.0
PATCH	You make backward-compatible bug fixes only — no new features.	1.4.2 → 1.4.3

3.2 Version Decision Examples

Concrete examples of how we decide which version number to increment:

Change	Version Bump	Reason
Fix the booking-overlap bug	1.0.0 → 1.0.1	Bug fix only
Add a "Cancel booking" feature	1.0.1 → 1.1.0	New feature, no breaking change
Add database index for performance	1.1.0 → 1.1.1	Internal improvement only
Add insurance Strategy pattern	1.1.1 → 1.2.0	New feature
Migrate password storage from SHA-256 to bcrypt	1.2.0 → 2.0.0	Breaking — existing hashes invalid
Add review system	2.0.0 → 2.1.0	New feature
Change the booking REST API URL	2.1.0 → 3.0.0	Breaking change

3.3 Git Tagging and Release Tracking

Every release is tagged in Git with the version number. This creates an immutable snapshot — anyone can check out exactly the code that was released, even years later:

```
# Create an annotated tag for the release
git tag -a v1.2.0 -m "Release 1.2.0 - Insurance feature"

# Push tags to remote repository
git push origin v1.2.0
```

```
# List all releases
git tag -l
```

3.4 CHANGELOG.md

We maintain a CHANGELOG.md file at the project root, following the "Keep a Changelog" format (keepachangelog.com). Every release lists what was Added, Changed, Deprecated, Removed, Fixed, and Security-related:

```
# Changelog

## [1.2.0] - 2026-03-15
### Added
- Insurance Strategy pattern with 3 plans (Basic, Standard, Premium)
- Discount code support (SAVE10, WELCOME20, SUMMER15)

### Changed
- Booking flow now requires insurance selection step

### Fixed
- Date overlap bug when bookings span month boundaries (#42)
- Login failure with email containing + character (#37)

## [1.1.0] - 2026-02-01
### Added
- Renter cancellation feature
- Admin user verification panel
```

4. Backward Compatibility Strategy

Backward compatibility means: *new versions of the software must not break existing users' data or workflows.* This is critical for a P2P platform where users have invested time creating listings, bookings, and reviews.

Without a backward compatibility plan, every upgrade would feel risky — users would resist upgrading, and the user base would fragment across incompatible versions. With a clear plan, upgrades become routine.

4.1 The Three Pillars of Backward Compatibility

Pillar	Goal	Example
Data Compatibility	Old data continues to work with the new version.	A booking created in v1.0 must still be readable in v2.0.
API Compatibility	Existing function signatures and command flags continue to work.	CarService.search_cars() still accepts the same arguments.
UX Compatibility	User workflows and menu structures remain familiar.	The "Make booking" menu option keeps its position and behavior.

4.2 Database Migration Strategy

The database schema will evolve over time (new columns, new tables, etc.). We handle this with **versioned migration scripts** — small, ordered SQL files that incrementally upgrade an existing database to a newer schema:

```
database/migrations/
001_initial_schema.sql # v1.0.0
002_add_insurance_column.sql # v1.2.0
003_add_reviews_table.sql # v1.3.0
004_add_loyalty_points.sql # v1.4.0
```

A migration runner script tracks which migrations have been applied (stored in a "schema_version" table) and applies any new ones on startup. This means:

- Users can upgrade from ANY old version to the latest — migrations chain automatically.
- Migrations are idempotent (safe to run twice).
- Each migration includes a rollback script in case of failure.
- Migration tests run in CI to ensure they work on real data samples.

4.3 API Deprecation Policy

Sometimes we need to remove or change a feature. Instead of breaking users immediately, we follow a graceful **deprecation lifecycle**:

Stage	Duration	What Happens
1. Announce	Major release	Method/feature marked @deprecated; warning logged when used.
2. Coexist	At least 6 months (1 minor cycle)	Old and new versions both work side by side; documentation steers users to new way.
3. Remove	Next major release only	Old method removed; users must have migrated by now.

Example in Python:

```
import warnings

def old_calculate_price(car, days):
    """DEPRECATED: Use BookingService.calculate_total() instead."""
    warnings.warn(
        "old_calculate_price() will be removed in v3.0. "
        "Use BookingService.calculate_total() instead.",
        DeprecationWarning, stacklevel=2
    )
    return car.daily_rate * days # Still works
```

4.4 Long-Term Support (LTS) Policy

Not every release deserves indefinite support. Our LTS policy clearly communicates what users can expect:

Release Type	Support Duration	What's Included
Current major	Active development	All updates + new features
Previous major	6 months after next major release	Security + critical bug fixes only
Older versions	No support	Users encouraged to upgrade

4.5 Configuration File Compatibility

When the config file format changes, we never silently break existing configs. Our config loader:

- Reads the config's "version" field to detect old formats.
- Auto-migrates old config files to the new format on first run.
- Backs up the original config file before any migration.
- Logs a warning so users know the format changed.

5. User Support Channels

Maintenance is only half the story — users also need a way to get help. We provide multiple support channels with clear expectations:

Channel	Best For	Response Time
In-app feedback form	Bug reports, feature requests, general feedback	24-48 hours
GitHub Issues	Technical bugs (for developers and power users)	1-3 business days
Email support@p2p-cars.com	Account issues, payment disputes, urgent matters	24 hours
Documentation site	How-to guides, FAQs, troubleshooting	Self-serve, 24/7
Community forum	Peer help, discussions, tips & tricks	Community-driven

5.1 Documentation Maintenance

Documentation drift — where docs become outdated relative to the code — is a chronic problem. We prevent this through:

Docs-as-code: Documentation lives in the same repo as code, and every pull request must update relevant docs.

Inline docstrings: Every public class and function has a docstring that auto-generates API documentation (using Sphinx).

Quarterly doc review: Every quarter, a designated team member walks through key docs from a new user's perspective and flags outdated sections.

Versioned docs: Each major version has its own documentation snapshot — users on v1 see v1 docs, not v2 docs.

6. Software Evolution Roadmap

The current v1.0 release covers core functionality. Here is the planned evolution over the next 24 months:

Version	Target Date	Highlights
v1.1.0	Q3 2026	Email notifications (SMTP integration); password reset flow.
v1.2.0	Q4 2026	Loyalty points system; renter tier discounts.
v1.3.0	Q1 2027	Multi-language support (English, Mongolian, Mandarin).
v2.0.0	Q2 2027	PostgreSQL backend; REST API for mobile clients (breaking change).
v2.1.0	Q3 2027	Mobile app (React Native) talking to v2 REST API.
v2.2.0	Q4 2027	Payment gateway integration (Stripe); automated fund disbursement to owners.
v3.0.0	Q2 2028	AI-based smart matching; predictive pricing.

6.1 The Role of Software Engineering Principles

The maintenance plan above is only achievable because we adhered to sound software engineering principles during the initial development:

Principle	How It Enables Maintenance
Encapsulation	Private attributes mean we can change internal storage without breaking callers. E.g. password hashing can switch from SHA-256 to bcrypt with no external API change.
Inheritance & Polymorphism	Adding a new role (e.g. "fleet operator") just requires a new subclass of User. Existing code that calls user.view_dashboard() works unchanged.
Abstraction	The InsuranceStrategy interface lets us add new insurance plans without touching the booking service.
Singleton (Pattern)	A single point of database access makes connection upgrades trivial — change DatabaseManager once, everywhere benefits.
Factory (Pattern)	User creation is centralized in UserFactory. Adding a new user type requires changes in ONE file, not scattered across the codebase.
Strategy (Pattern)	New pricing algorithms or insurance plans are added as new Strategy classes — open/closed principle in action.
Layered architecture	UI, services, models, and database are separated. We can replace the CLI with a web UI without touching business logic.
Configuration externalization	Environment-specific values live in config.py. Switching between dev/test/prod requires no code changes.

7. Conclusion

A maintenance plan is not a document you write once and forget — it is a living strategy that evolves with the software. This plan establishes:

Software maintenance: A structured approach to corrective, adaptive, perfective, and preventive maintenance, supported by clear SLAs and a predictable release cadence.

Versioning: Semantic Versioning (MAJOR.MINOR.PATCH) with Git tags and a Keep-a-Changelog formatted CHANGELOG.md so every release is traceable.

Backward compatibility: Versioned database migrations, a deprecation lifecycle that gives users 6+ months to adapt, and Long-Term Support for critical legacy versions.

The P2P Car Rental System was designed from the ground up with maintenance in mind — using OOP principles, design patterns, layered architecture, and externalized configuration. These choices were not aesthetic; they directly enable the maintenance practices described in this document. A system designed without these principles would make even routine maintenance painful and risky.

By following this plan, we ensure the P2P Car Rental System remains reliable, secure, and pleasant to use for years to come — protecting the investment of both our users and our development team.

Document Version	1.0.0
Last Updated	May 2026
Next Review	November 2026
Approved By	Development Lead